

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 11

Scheduling:
concepts and classic policies

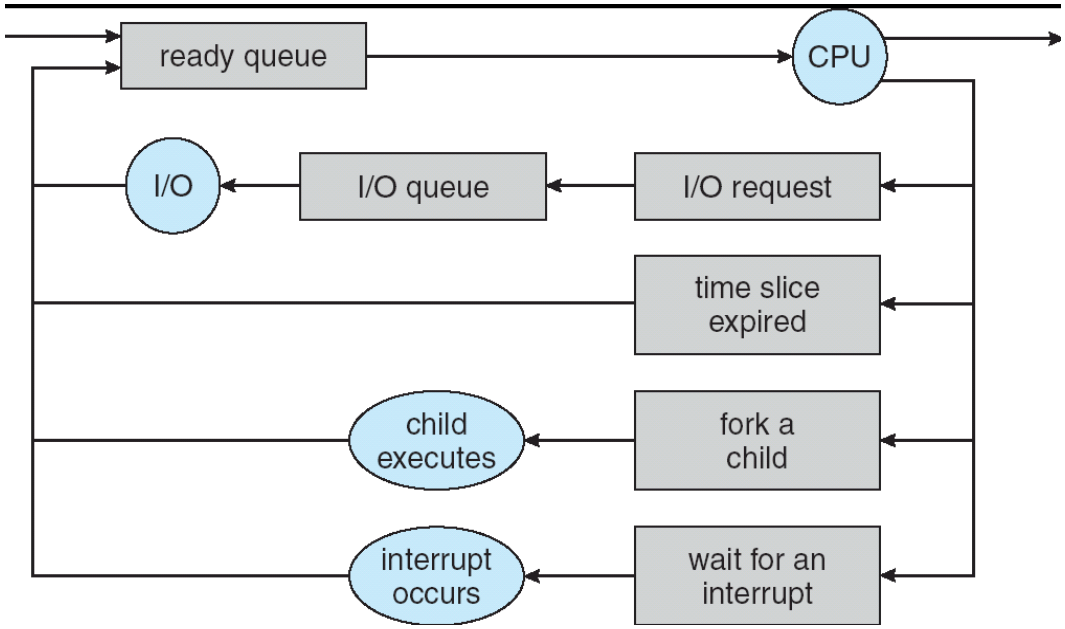
Prof. Yunming XIAO

School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

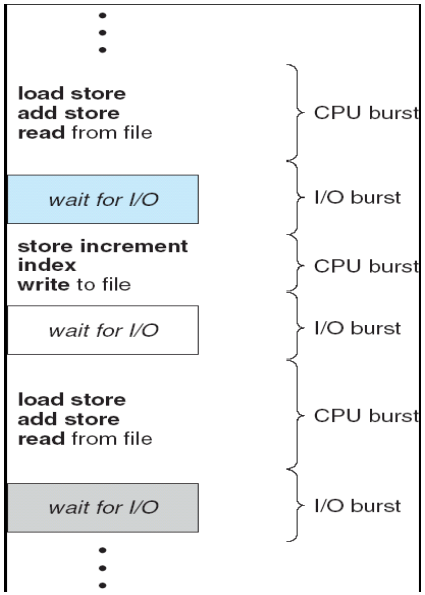
Basic scheduling

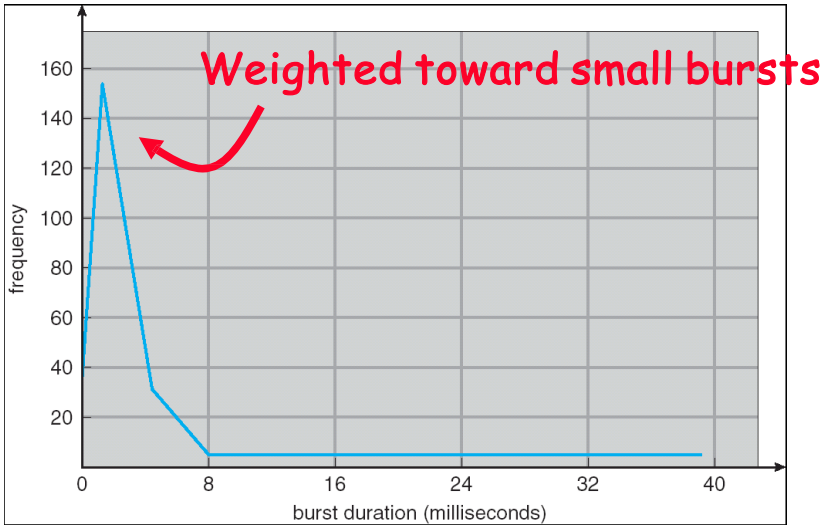
Scheduling



- Question: how is the OS to decide which of several tasks to take off a queue?
- Scheduling: deciding which threads are given access to resources from moment to moment
  - Often, we think in terms of CPU time, but could also think about access to resources like network bandwidth or disk access

Assumption: CPU bursts





- Execution model: programs alternate between bursts of CPU and I/O
  - Program typically uses the CPU for some period of time, then does I/O, then uses CPU again
  - Each scheduling decision is about which job to give to the CPU for use by its next CPU burst
  - With timeslicing, thread may be forced to give up CPU before finishing current CPU burst

Scheduling policy goals/criteria

- Minimize response time
  - Minimize elapsed time to do an operation (or job)
  - Response time is what the user sees:
    - Time to echo a keystroke in editor
    - Time to compile a program
    - Real-time tasks: must meet deadlines imposed by “world”
- Maximize throughput
  - Maximize operations (or jobs) per second
  - Throughput related to response time, but not identical:
    - Minimizing response time will lead to more context switching than if you only maximized throughput
  - Two parts to maximizing throughput
    - Minimize overhead (for example, context-switching)
    - Efficient use of resources (CPU, disk, memory, etc)

Scheduling policy goals/criteria

- Fairness
  - Share CPU among users in some equitable way
  - Fairness is not minimizing average response time:
    - Better average response time by making system less fair

First-Come First-Served (FCFS) scheduling

- First-Come, First-Served (FCFS)
  - Also “First In, First Out” (FIFO) or “Run until done”
    - In early systems, FCFS meant one program scheduled until done (including I/O)
    - Now, it means keep CPU until thread blocks

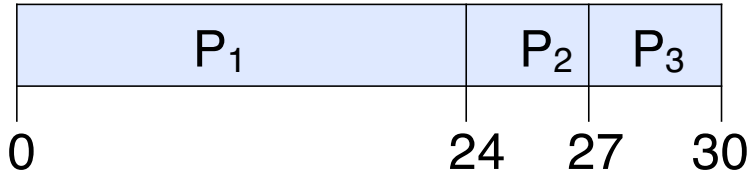


First-Come First-Served (FCFS) scheduling

- Example:

Process	Burst Time
P_1	24
P_2	3
P_3	3

  - Suppose processes arrive in the order: P1 , P2 , P3
  - The Gantt Chart for the schedule is:



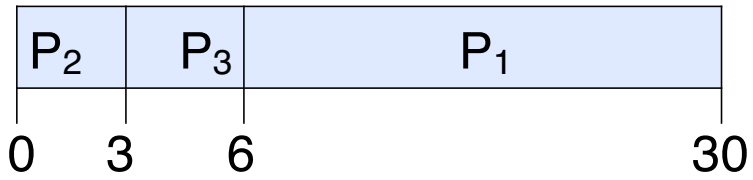
  - Waiting time for P1 = 0; P2 = 24; P3 = 27
  - Average waiting time: (0 + 24 + 27)/3 = 17
  - Average completion time: (24 + 27 + 30)/3 = 27
- *Convoy effect*: short process stuck behind long process

First-Come First-Served (FCFS) scheduling

- Example continued:

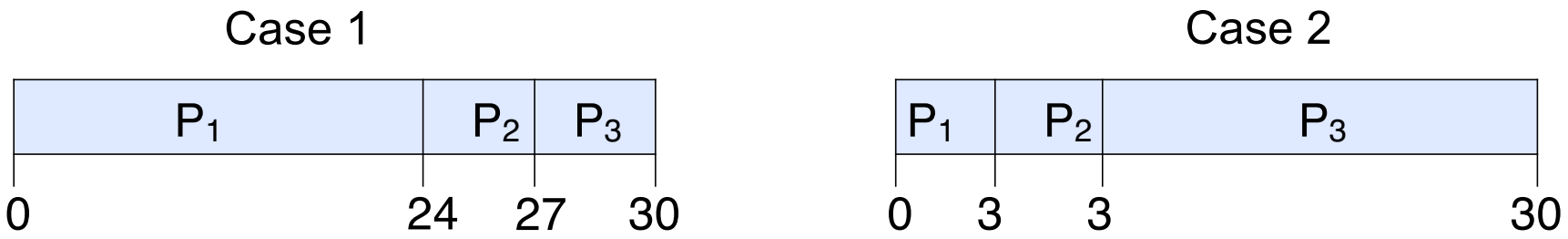
Process	Burst Time
P_2	3
P_3	3
P_1	24

  - Suppose processes arrive in the order: P2 , P3 , P1
  - Now, the Gantt Chart for the schedule is:



  - Waiting time for P1 = 6; P2 = 0; P3 = 3
  - Average waiting time: (6 + 0 + 3)/3 = 3
  - Average completion time: (3 + 6 + 30)/3 = 13

First-Come First-Served (FCFS) scheduling



- In second case:
  - Average waiting time is much better (before it was 17)
  - Average completion time is better (before it was 27)
- FIFO Pros and Cons:
  - Simple (+)
  - Short jobs get stuck behind long ones (-)
    - Convenience store: getting milk, always stuck behind cart full of items!!

Round Robin (RR) scheduling

- FCFS Scheme: potentially bad for short jobs!
  - Depends on submit order
  - If you are first in line at supermarket with milk, you don't care who is behind you, on the other hand…
- Round Robin scheme: Preemption!
  - Each process gets a small unit of CPU time (time quantum), usually 10-100 milliseconds
  - After quantum expires, the process is preempted and added to the end of the ready queue
  - n processes in ready queue and time quantum is q ⇒
    - Each process gets 1/n of the CPU time
    - In chunks of at most q time units
    - No process waits more than (n-1)q time units

Round Robin (RR) scheduling

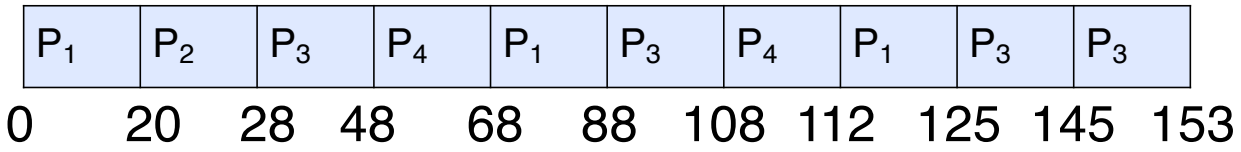
- Performance
  - q large ⇒ FCFS
  - q small ⇒ Interleaved (really small ⇒ hyperthreading?)
  - q must be large with respect to context switch, otherwise overhead is too high (all overhead)

Round Robin (RR) scheduling

- Example continued:

Process	Burst Time
P_1	53
P_2	8
P_3	68
P_4	24

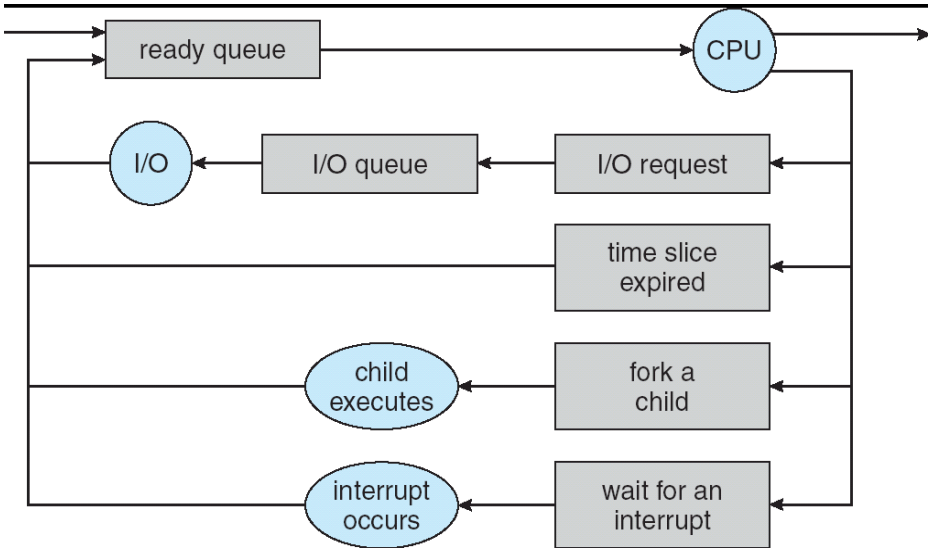
  - The Gantt Chart for the schedule is:



  - Waiting time: P1=(68-20)+(112-88)=72; P2=(20-0)=20
    P3=(28-0)+(88-48)+(125-108)=85; P4=(48-0)+(108-68)=88
  - Average waiting time: (72 + 20 + 85 + 88)/4 = 66.25
  - Average completion time: (125 + 28 + 153 + 112)/4 = 104.25
- Thus, Round-Robin Pros and Cons:
  - Better for short jobs, Fair (+)
  - Context-switching time adds up for long jobs

How to implement RR in the kernel?

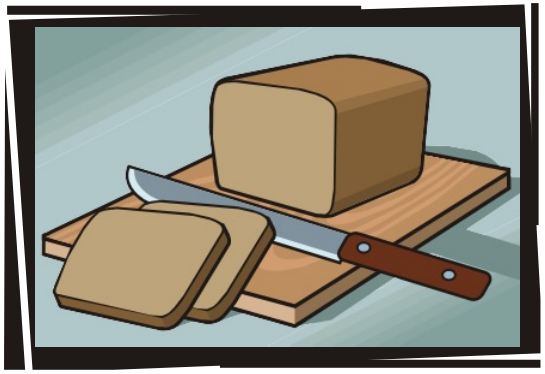
- FIFO queue, as in FCFS
- But preempt job after quantum expires, and send it to the back of the queue
  - How? Time interrupt!
  - And, of course, careful synchronization



Project 2: Scheduling

RR discussion

- How do you choose time slice?
  - What if too big?
    - Response time suffers
  - What if infinite (∞)?
    - Get back FIFO
  - What if time slice too small?
    - Throughput suffers!



RR discussion

- Actual choices of timeslice:
  - Initially, UNIX timeslice one second:
    - Worked ok when UNIX was used by one or two people.
    - What if three compilations going on?

      3 seconds to echo each keystroke!
  - Need to balance short-job performance and long-job throughput:
    - Typical context-switching overhead is 0.1ms – 1ms
    - Typical time slice today is between 10ms – 100ms
    - Roughly 1% overhead due to context-switching

Comparisons between FCFS and RR

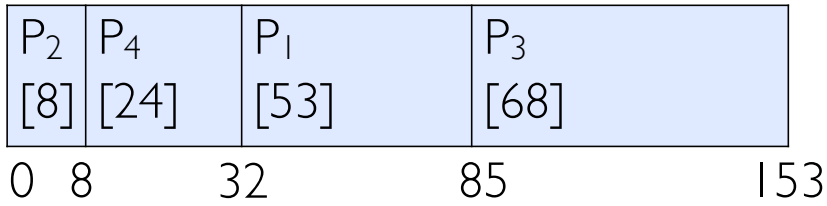
- Assuming zero-cost context-switching time, is RR always better than FCFS?
- Simple example: 10 jobs, each take 100s of CPU time
  RR scheduler quantum of 1s
  All jobs start at the same time
- Completion times:

Job #	FIFO	RR
1	100	991
2	200	992
…	…	…
9	900	999
10	1000	1000

  - Both RR and FCFS finish at the same time
  - Average completion time is much worse under RR!
    - Bad when all jobs are the same length

Earlier example with different time quantum

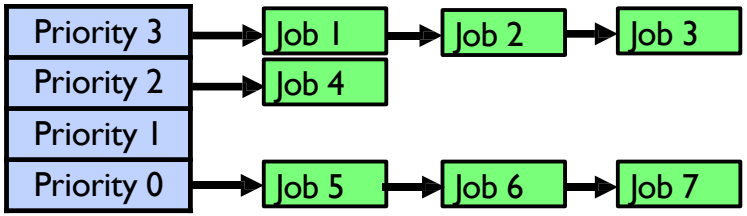
Best FCFS:



	Quantum	P_1	P_2	P_3	P_4	Average
Wait Time	Best FCFS	32	0	85	8	31¼
	Q = 1	84	22	85	57	62
	Q = 5	82	20	85	58	61¼
	Q = 8	80	8	85	56	57¼
	Q = 10	82	10	85	68	61¼
	Q = 20	72	20	85	88	66¼
	Worst FCFS	68	145	0	121	83½
Completion Time	Best FCFS	85	8	153	32	69½
	Q = 1	137	30	153	81	100½
	Q = 5	135	28	153	82	99½
	Q = 8	133	16	153	80	95½
	Q = 10	135	18	153	92	99½
	Q = 20	125	28	153	112	104½
	Worst FCFS	121	153	68	145	121¾

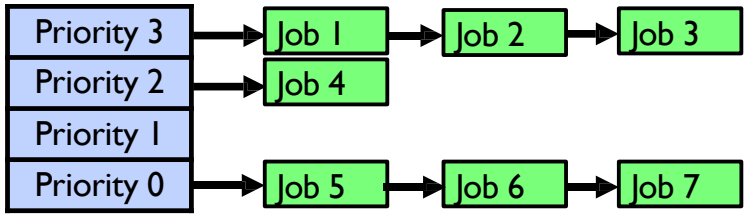
Priority and fairness

Handling differences in importance: strict priority scheduling



- Execution plan
  - Always execute highest-priority runnable jobs to completion
  - Each queue can be processed in RR with some time-quantum
- Problems:
  - Starvation: lower priority jobs don't get to run because higher priority jobs
  - Deadlock: priority inversion
    - Happens when low priority task has lock needed by high-priority task
    - Usually involves third, intermediate priority task preventing high-priority task from running

Handling differences in importance: strict priority scheduling



- How to fix problems?
  - Dynamic priorities – adjust base-level priority up or down based on heuristics about interactivity, locking, burst behavior, etc…

Scheduling fairness

- What about fairness?
  - Strict fixed-priority scheduling between queues is unfair (run highest, then next, etc):
    - Long running jobs may never get CPU
    - Urban legend: in Multics, shut down machine, found 10-year-old job
      ⇒ Ok, probably not…
  - Must give long-running jobs a fraction of the CPU even when there are shorter jobs to run
  - Tradeoff: fairness gained by hurting average response time!

Scheduling fairness

- How to implement fairness?
  - Could give each queue some fraction of the CPU
    - What if one long-running job and 100 short-running ones?
    - Like express lanes in a supermarket—sometimes express lanes get so long, get better service by going into one of the other lines
  - Could increase priority of jobs that don't get service
    - What is done in some variants of UNIX
    - This is ad hoc—what rate should you increase priorities?
    - And, as system gets overloaded, no job gets CPU time, so everyone increases in priority ⇒ interactive jobs suffer

What if we knew the future?



What if we knew the future?



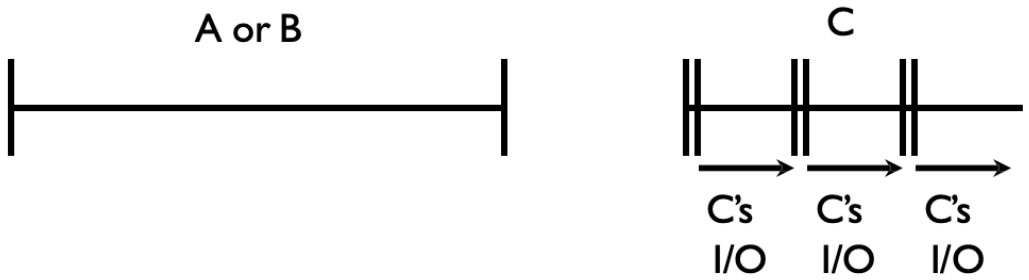
- Could we always mirror best FCFS?
- Shortest Job First (SJF):
  - Run whatever job has least amount of computation to do
  - Sometimes called “Shortest Time to Completion First” (STCF)
- Shortest Remaining Time First (SRTF):
  - Preemptive version of SJF: if job arrives and has a shorter time to completion than the remaining time on the current job, immediately preempt CPU
  - Also called “Shortest Remaining Time to Completion First” (SRTCF)
- These can be applied to whole program or current CPU burst
  - Idea is to get short jobs out of the system
  - Big effect on short jobs, only small effect on long ones
  - Result is better average response time

Discussion

- SJF/SRTF are the best you can do at minimizing average response time
  - Provably optimal (SJF among non-preemptive, SRTF among preemptive)
  - Since SRTF is always at least as good as SJF, focus on SRTF

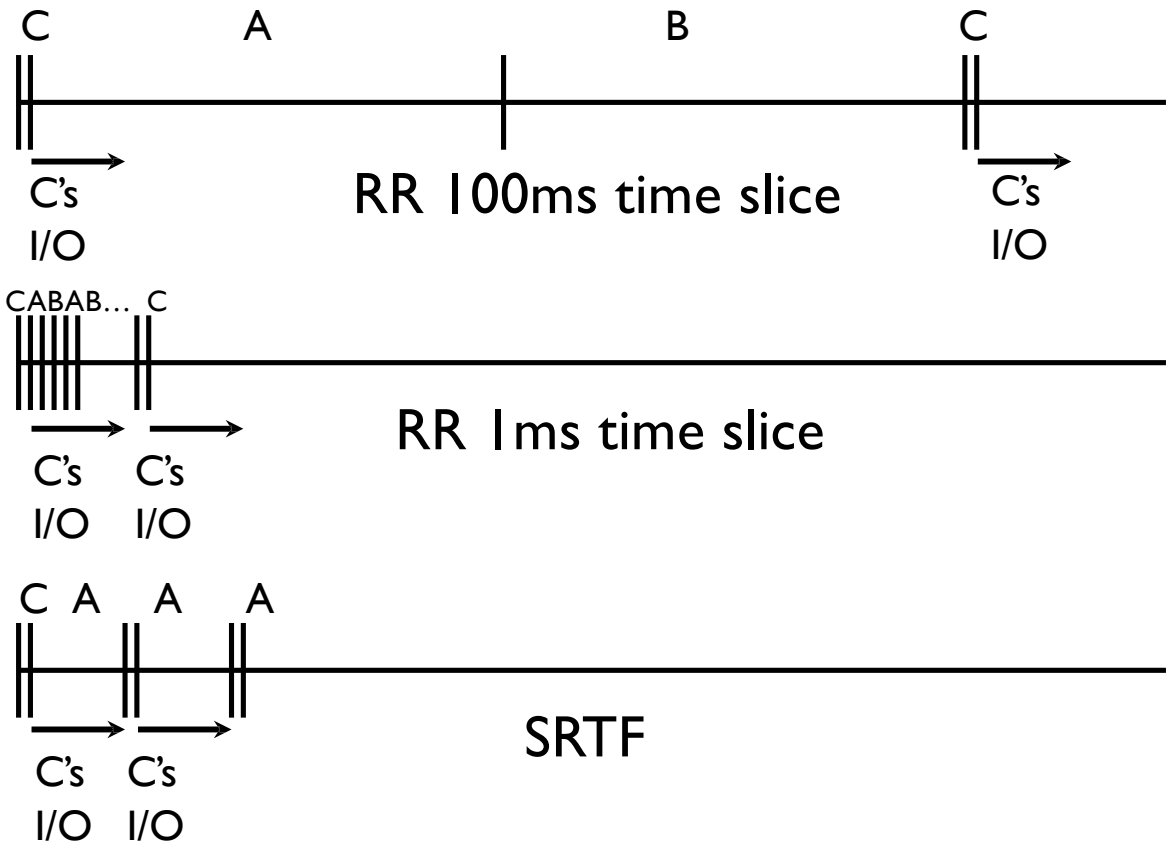
- Comparison of SRTF with FCFS
  - What if all jobs the same length?
    - SRTF becomes the same as FCFS (i.e. FCFS is best can do if all jobs the same length)
  - What if jobs have varying length?
    - SRTF: short jobs not stuck behind long ones

Example to illustrate benefits of SRTF

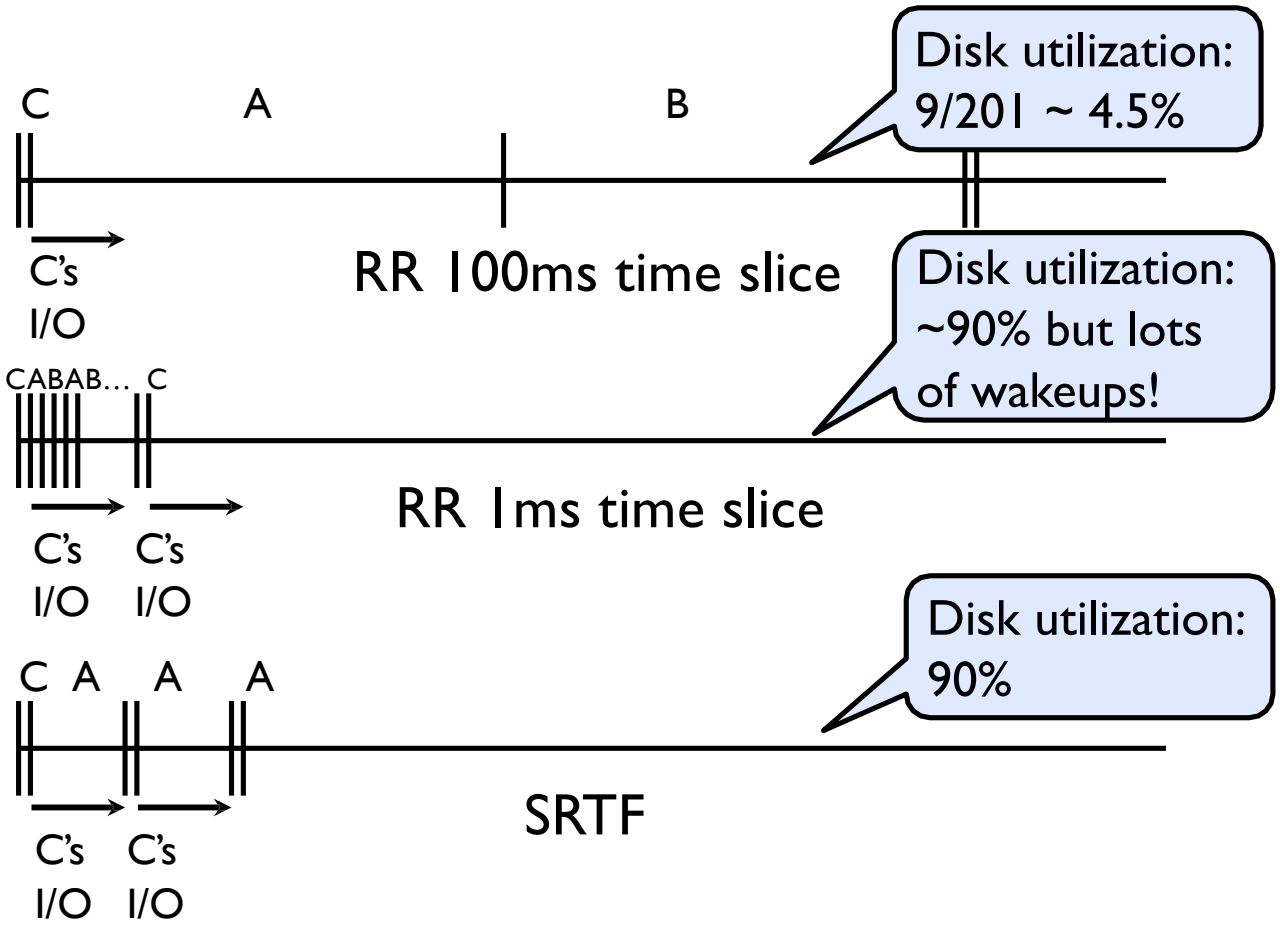


- Three jobs:
  - A, B: both CPU bound, run for week
    C: I/O bound, loop 1ms CPU, 9ms disk I/O
  - If only one at a time, C uses 90% of the disk, A or B could use 100% of the CPU
- With FCFS:
  - Once A or B get in, keep CPU for two weeks
- What about RR or SRTF?
  - Easier to see with a timeline

SRTF example continued



SRTF example continued



More discussion about SRTF

- Starvation
  - SRTF can lead to starvation if many small jobs!
  - Large jobs never get to run
- Somehow need to predict future
  - How can we do this?
  - Some systems ask the user
    - When you submit a job, have to say how long it will take
    - To stop cheating, system kills job if takes too long
  - But: hard to predict job's runtime even for non-malicious users



More discussion about SRTF

- Bottom line, can't really know how long job will take
  - However, can use SRTF as a yardstick for measuring other policies
  - Optimal, so can't do any better
- SRTF Pros & Cons
  - Optimal (average response time) (+)
  - Hard to predict future (-)
  - Unfair (-)



Predicting the future

Predicting the length of the next CPU burst

- Adaptive: changing policy based on past behavior
  - CPU scheduling, in virtual memory, in file systems, etc
  - Works because programs have predictable behavior
    - If program was I/O bound in past, likely in future
    - If computer behavior were random, wouldn't help

Predicting the length of the next CPU burst

- Example: SRTF with estimated burst length
  - Use an estimator function on previous bursts:

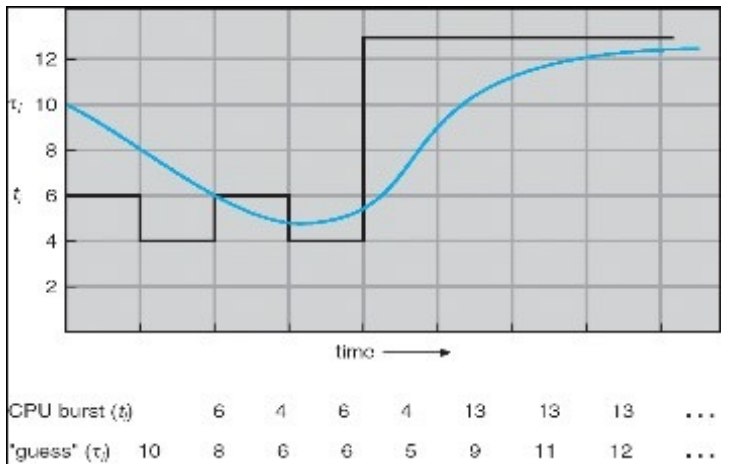
    Let t_{n-1}, t_{n-2}, t_{n-3}, etc. be previous CPU burst lengths.

    Estimate next burst $t_n = f(t_{n-1}, t_{n-2}, t_{n-3}, \ldots)$
  - Function f could be one of many different time series estimation schemes (Kalman filters, etc)
  - For instance,

Exponential averaging

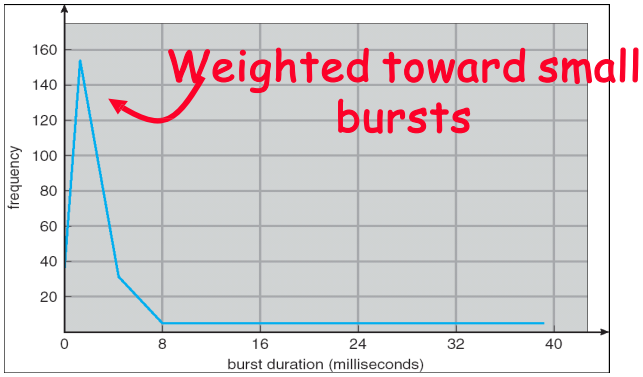
$t_n = \alpha t_{n-1} + (1-\alpha) t_{n-2}$

with $(0 < \alpha < 1)$



How to handle simultaneous mix of different types of apps?

- Consider mix of interactive and high throughput apps:
  - How to best schedule them?
  - How to recognize one from the other?
    - Do you trust app to say that it is “interactive”?
  - Should you schedule the set of apps identically on servers, workstations, pads, and cellphones?
- For instance, is Burst Time (observed) useful to decide which application gets CPU time?
  - Short Bursts ⇒ Interactivity ⇒ High Priority?



Conclusion

- Scheduling goals:
  - Minimize Response Time (e.g. for human interaction)
  - Maximize Throughput (e.g. for large computations)
  - Fairness (e.g. Proper Sharing of Resources)
  - Predictability (e.g. Hard/Soft Realtime)
- Round-Robin scheduling:
  - Give each thread a small amount of CPU time when it executes; cycle between all ready threads
  - Pros: Better for short jobs
- Shortest Job First (SJF) / Shortest Remaining Time First (SRTF):
  - Run whatever job has the least amount of computation to do/least remaining amount of computation to do

Reading for next lecture

- Operating System Concepts:
  - Chapter 5.6 (5.1-5.3 if you want to review today's content)

Or

- Operating System: Three Easy Pieces:
  - Chapters 8-9 (7-8 if you want to review today's content)

Thanks